

ASSIGNMENT 3

U24AI044

NISHCHAY DEEPAK MITTAL

1. Implement a Simple Reflex Agent for a vacuum cleaner that can navigate a three-room environment (A, B, and C) and maintain cleanliness. Identify the “rationality” of its actions. How will you define the performance cost? Define the complete Rule Table and store it either in a dictionary/vector/matrix based on the language you implement the vacuum cleaner. Show the simulation output with percept, action, and location. Do you need priorities in the rules?

Each room is either clean or dirty, the agents checks in which room it is in and whether it is clean or dirty and then selects the best action based on the given conditions

Percept - Room, Clean/Dirty

Action – clean, move right, move left

RATIONALITY OF ACTION

An agent is rational if it always chooses action that maximizes the performance measure based on current input.

The given vacuum agent is rational because:

- if the room is dirty,it cleans immediately
- if room is clean,it moves to different room
- it never stays idle in any room
- it cleans all rooms

It cleans whenever dirt is detected and moves only when necessary, ensuring optimal behavior under the given performance measure.

PERFORMANCE COST

The performance cost is defined as the total number of actions performed by the agent.

Cleaning action cost=5

Move Right = 1

Move Left = 1

Goal is to clean all rooms with minimum total actions.

RULE TABLE

('A', 'Dirt'): ['Remove']

('A', 'No Dirt'): ['Move Right'],

('B', 'Dirt'): ['Remove'],

('B', 'No Dirt'): ['Move Left', 'Move Right'],

('C', 'Dirt'): ['Remove'],

('C', 'No Dirt'): ['Move Left']

IS PRIORITY NEEDED?

YES, priority is needed

The highest priority – If the room is dirty then clean it

Lower priorities – movement if the room is clean

```
import random

class Env:
    def __init__(self):
        self.rooms = {
            'A': 'Dirt',
            'B': 'Dirt',
            'C': 'Dirt'
        }

    def stat(self, loc):
        return self.rooms[loc]

    def clean(self, loc):
        self.rooms[loc] = 'No Dirt'

class Agent:
    def __init__(self, envi):
        self.env = envi
        self.loc = 'A'
        self.tc = 0

        self.rule = {
            ('A', 'Dirt'): ['Remove'],
            ('A', 'No Dirt'): ['Move Right'],
            ('B', 'Dirt'): ['Remove'],
            ('B', 'No Dirt'): ['Move Left', 'Move Right'],
            ('C', 'Dirt'): ['Remove'],
            ('C', 'No Dirt'): ['Move Left']
        }
```

```

    }

    def move(self, action):
        if self.loc == 'A' and action == 'Move Right':
            self.loc = 'B'
        elif self.loc == 'B' and action == 'Move Left':
            self.loc = 'A'
        elif self.loc == 'B' and action == 'Move Right':
            self.loc = 'C'
        elif self.loc == 'C' and action == 'Move Left':
            self.loc = 'B'

        self.tc += 1

    def act(self):
        status = self.env.stat(self.loc)
        action = random.choice(self.rule[(self.loc, status)])

        print(f"({self.loc},{status})\t{action}\t\tCost:{self.tc}")

        if action == 'Remove':
            self.env.clean(self.loc)
            self.tc += 5
        else:
            self.move(action)

env = Env()
agent = Agent(env)

print("Percept\t\tAction\t\tPerformance")

for _ in range(20):
    agent.act()

print("\nTotal Performance Cost:", agent.tc)

```

OUTPUT

```
PS C:\Users\mitta\OneDrive\Desktop\AI Lab\Lab3> python
Percept      Action      Performance
(A,Dirt)      Remove      Cost:0
(A,No Dirt)   Move Right   Cost:5
(B,Dirt)      Remove      Cost:6
(B,No Dirt)   Move Left    Cost:11
(A,No Dirt)   Move Right   Cost:12
(B,No Dirt)   Move Right   Cost:13
(C,Dirt)      Remove      Cost:14
(C,No Dirt)   Move Left    Cost:19
(B,No Dirt)   Move Right   Cost:20
(C,No Dirt)   Move Left    Cost:21
(B,No Dirt)   Move Right   Cost:22
(C,No Dirt)   Move Left    Cost:23
(B,No Dirt)   Move Right   Cost:24
(C,No Dirt)   Move Left    Cost:25
(B,No Dirt)   Move Left    Cost:26
(A,No Dirt)   Move Right   Cost:27
(B,No Dirt)   Move Right   Cost:28
(C,No Dirt)   Move Left    Cost:29
(B,No Dirt)   Move Right   Cost:30
(C,No Dirt)   Move Left    Cost:31

Total Performance Cost: 32
```

2. Design a Simple Reflex Agent for an Indian Railways Level Crossing. The agent must ensure maximum safety for road traffic while minimizing delays for the train, using multi-source sensory input without historical tracking.

The environment consists of a double-track railway line intersecting a busy state highway. The percepts include the following:

- **Track Sensors (Inbound/Outbound):** Sensors placed 2km away that detect if a train is currently passing over them. It is either detected or not detected.
- **Obstacle Sensors (Yellow Box):** A grid of sensors on the road between the gates

to detect “stuck” vehicles (e.g., an auto-rickshaw or buffalo).

- **Manual Emergency Input:** A physical lever for the station master. It is either Neutral or Active.

Actuators:

- **Gate Arm:** Lower/Raise
- **Hooter/Siren:** On/Off
- **Signal to Train:** Green (Safe to proceed)/Red (Emergency Stop)

Design the rules set. Show the simulation output with percept, action, and location. Do you need priorities in the rules?

RULE SET

1)Emergency - True

Gate – lower

Siren – on

Train signal – red

2)Obstacle – detected

Emergency – neutral

Gate – raise

Siren – on

Train signal – red

3)Train incoming – detected

Obstacle – clear

Emergency – neutral

Gate- lower

Siren – on

Train signal – green

4)Train incoming – not detected

Obstacle – clear

Emergency – neutral

Gate – raise

Siren – off

Train signal – green

IS PRIORITY NECESSARY?

YES, priority required

Order –

- 1) Emergency Override
- 2) Obstacle
- 3) Train detected
- 4) All clear

```
class Agent:

    def __init__(self):
        pass

    def get_action(self, track, obstacle, emergency):
        """
        track: 0/1 -> Train detected or not
        obstacle: 0/1 -> Vehicle stuck or not
        emergency: 0/1 -> Manual emergency lever
        """

        # RULE 1: Emergency Override
        if emergency == 1:
            return {
                "Gate": "Lower",
                "Siren": "ON",
                "TrainSignal": "RED",
                "Reason": "Emergency Lever Activated"
            }

        # RULE 2: Obstacle Detected
        if obstacle == 1:
            return {
                "Gate": "Raise",
                "Siren": "ON",
                "TrainSignal": "RED",
                "Reason": "Obstacle Detected"
            }

        # RULE 3: Train Approaching
```

```

        if track == 1:
            return {
                "Gate": "Lower",
                "Siren": "ON",
                "TrainSignal": "RED",
                "Reason": "Train Approaching"
            }

        # RULE 4: Safe Condition
        return {
            "Gate": "Raise",
            "Siren": "OFF",
            "TrainSignal": "GREEN",
            "Reason": "Safe Condition"
        }

agent = Agent()

test_inputs = [
    (0, 0, 0),
    (1, 0, 0),
    (0, 1, 0),
    (1, 1, 0),
    (0, 0, 1),
    (1, 0, 1),
    (0, 1, 1),
    (1, 1, 1)
]

print("T O E -> Gate | Siren | Train Signal | Reason")

for t, o, e in test_inputs:
    action = agent.get_action(t, o, e)
    print(f"{t} {o} {e} -> {action['Gate']} | {action['Siren']} | "
          f"{action['TrainSignal']} | {action['Reason']}")

```

OUTPUT

```
PS C:\Users\mitta\OneDrive\Desktop\AI Lab\Lab3> python
T O E -> Gate | Siren | Train Signal | Reason
0 0 0 -> Raise | OFF | GREEN | Safe Condition
1 0 0 -> Lower | ON | RED | Train Approaching
0 1 0 -> Raise | ON | RED | Obstacle Detected
1 1 0 -> Raise | ON | RED | Obstacle Detected
0 0 1 -> Lower | ON | RED | Emergency Lever Activated
1 0 1 -> Lower | ON | RED | Emergency Lever Activated
0 1 1 -> Lower | ON | RED | Emergency Lever Activated
1 1 1 -> Lower | ON | RED | Emergency Lever Activated
```